

CloudBees CI: Configuration as Code (CasC) — Official Implementation Runbook

Scenario: an existing CloudBees CI installation (operations center + one or more managed controllers) that was set up and configured manually through the UI, with no CasC in place yet. This runbook covers exporting the existing configuration into CasC bundles, sanitizing them for git, enabling CasC on the operations center, choosing and enabling a mechanism to feed managed controllers their bundles, bringing existing and brand-new controllers under CasC, and the operational gotchas encountered doing this for real (crash loops, silent misconfiguration, cluster capacity limits).

Follow the phases in order. Do not skip Phase 2/2.5 sanitization — committing a raw export to git commits license keys and encrypted credentials.

Prerequisites

- `kubectl` access to the cluster, `helm` with the `cloudbees` repo added (<https://charts.cloudbees.com/public/cloudbees>), `git`, `python3`
- A CJOC admin user with an API token, and admin access to each managed controller you'll export from
- A git repo to hold the CasC bundle(s) — public or private; if private, you'll need SSH or username/password credentials for the SCM retriever later. Your git host's branch-protection policy may block direct pushes to `main` — plan to work in feature branches + PRs (see Phase 6).
- Required plugins (verify, don't assume — check via `<instance-url>/pluginManager/api/json?depth=1&tree=plugins[shortName,version,enabled]`):
 - `configuration-as-code` (Jenkins core CasC engine)
 - `cloudbees-casc-client` (v1.44+ / CloudBees CI 2.319.1.5+)
 - `cloudbees-casc-shared`, `cloudbees-casc-server`, `cloudbees-casc-items-api`, `cloudbees-casc-items-server`
 - `cloudbees-casc-items-controller` — specifically needed on managed controllers for item (job/folder) creation via CasC
 - These ship bundled with CloudBees CI on modern cloud platforms in practice — this check is a formality, not usually a real blocker
- Know your CloudBees license situation: a purchased (non-trial) license's cert/key will show up in the **operations center** export and needs to be handled as a secret (Phase 2). Managed controller exports do **not** need this — see Phase 2.5.

Phase 1 — Get the existing configuration as a bundle

Two ways to export the *current, live* configuration into CasC bundle files — use this instead of hand-writing YAML from scratch. Do this against **each instance separately** — the operations center's own URL for the OC bundle, and each managed controller's own URL for its bundle.

UI: Manage Jenkins → **CloudBees Configuration as Code export and update** → **Current configuration** → **Download** (zip or individual files)

API (scriptable, same result):

```
Shell
curl -gS -u "<user>:<token>" -o casc-export.txt \
  "https://<instance-url>/core-casc-export"
```

Note: the plain endpoint (`/core-casc-export`) returns a redirect — use `-L` to follow it, or use `/core-casc-export/zipExport` for a zip if your version supports it (availability varies by version — if it 404s, fall back to the UI download, which always works).

Unzip/split the result — you'll get some subset of:

- `bundle.yaml` (manifest — lists which of the below are included)
- `jenkins.yaml` (the actual JCasC config — security realm, credentials, license, system settings, tool installs)
- `plugins.yaml` (plugin list)
- `rbac.yaml` (groups/roles, if using matrix-based RBAC)
- `items.yaml` (meaning differs by instance type — see Phase 2.5)
- `plugin-catalog.yaml` (managed controllers only — see Phase 2.5)

Phase 2 — Sanitize the bundle before it goes anywhere near git

Do not commit the raw export. CloudBees's own docs are explicit that raw exports "should not be used directly." This phase covers the **operations center** bundle specifically — see Phase 2.5 for managed-controller differences.

2a. Secrets (hard blockers)

- Any `credentials.system.domainCredentials` entries in `jenkins.yaml` have passwords encrypted with this instance's `master.key` — not portable, and shouldn't be committed even encrypted. Replace with a variable placeholder: `password: "${SOME_CREDENTIAL_PASSWORD}"`.
- For the **operations center** (or a client controller) bundle, the export will include a `license:` block with a full certificate and RSA private key in plaintext — the single most sensitive thing in the export. Replace both with placeholders:

None

```
license:
  certificate: "${CJOC_LICENSE_CERT}"
  key: "${CJOC_LICENSE_KEY}"
```

- Supplying the real values: create a Kubernetes Secret with these as keys, then reference it via `OperationsCenter.ContainerEnvFrom` in Helm (exact mechanism in Phase 5) — this injects the Secret's keys as environment variables into the CJOC container, resolving `${VAR}` placeholders at startup. **Never** put real cert/key/password values in any file that lands in a git working directory — write them to a location outside any git working directory, delete them once the Secret is created.

2b. Cosmetic but recommended

- `bundle.yaml`: give it a real `id` and `description` (the export defaults to generic placeholders like `"jenkins"` / `"Autogenerated bundle descriptor"`)
- Review `unclassified.location.url` / `adminAddress` and any other embedded hostname — for a bundle describing this specific instance, keeping the real hostname is usually correct; flag it as a conscious decision either way.

2c. Validation errors you will likely hit — fix these proactively

1. **rbac.yaml schema error**: `"required property 'groups' not found"` / `"required property 'roles' not found"`. The raw export often produces a nearly-empty `rbac.yaml` (just `removeStrategy: {rbac: SYNC}`) that fails schema validation.
 - If the instance genuinely doesn't use matrix-based RBAC (check `jenkins.authorizationStrategy` in `jenkins.yaml` — `loggedInUsersCanDoAnything` rather than a matrix strategy), the cleanest fix is to **remove the `rbac:` entry from `bundle.yaml` entirely**, not patch `rbac.yaml`'s content. Including any `rbac.yaml` in the manifest triggers CasC's RBAC configurator, which then *requires*

`RoleMatrixAuthorizationStrategy` to be the active strategy — a second, harder failure (`RoleMatrixAuthorizationStrategy` is not the current authorization strategy and it is required). Adding empty `groups: []/roles: []` passes the schema check but walks straight into this — don't.

- If the instance does use matrix RBAC, populate `groups/roles` for real from what's actually configured.
2. **advisor block validation error**: an empty `advisor: {}` fails with `ConfiguratorException: advisor: Invalid configuration for CloudBees Jenkins Advisor`. This is CloudBees's opt-in health/usage reporting — a real decision, not just plumbing:
- To disable: remove the `advisor:` key from `jenkins.yaml` entirely.
 - To enable (ask the customer first — it opts into sending data to CloudBees): requires both fields:

None

```
advisor:
  acceptToS: true
  email: "someone@customer.com"
```

3. **"workflow-cps is not in the CloudBees Assurance Program (CAP)"** — a warning at the OC level, but a real installation problem on a managed controller if not addressed — see Phase 2.5's `plugin-catalog.yaml` explanation.
-

Phase 2.5 — Managed controller bundles are not the same shape as the OC bundle

Do the export + sanitize pass separately for each managed controller, against **that controller's own URL**, not CJOC's.

File list differences

- `bundle.yaml` and `jenkins.yaml` required, same as OC.
- `plugins.yaml`, `items.yaml`, `rbac.yaml`, `variables.yaml` optional, same as OC.
- **`plugin-catalog.yaml` is controller-specific** — doesn't apply to OC bundles. Two separate plugin mechanisms:
 - `plugins.yaml` — CloudBees Assurance Program (CAP) plugins, bundled directly into the controller image, no internet connectivity needed.
 - `plugin-catalog.yaml` — plugins **outside** CAP, downloaded from Update Centers or a custom URL at provisioning time. Any plugin listed here (and its

dependencies) must **also** be listed in `plugins.yaml` — the catalog says where to get it from, not whether to install it. This is exactly what the "workflow-cps is not in CAP" warning from 2c means in practice.

`items.yaml` means something different here

- OC's `items.yaml` declares **managed controllers themselves** (image, resources, storage class — `kind: managedController` entries, one per controller).
- A controller's own `items.yaml` declares **that controller's actual work**: folders, freestyle jobs, pipelines, multibranch pipelines, organization folders. Example shape:

```
None
removeStrategy:
  items: "none"          # or "remove-all"
  rbac: "sync"
items:
- kind: "folder"
  name: "my-folder"
  displayName: "My Project Folder"
- kind: "pipeline"
  name: "my-pipeline"
  definition:
    cpsScmFlowDefinition:
      scriptPath: "Jenkinsfile"
      scm:
        gitSCM:
          userRemoteConfigs:
            - userRemoteConfig:
                url: "git@github.com:org/repo.git"
          branches:
            - branchSpec:
                name: "*/main"
```

- Item creation via CasC on a controller needs the `cloudbees-casc-items-controller` plugin (Prerequisites).

License: remove, don't substitute

Per CloudBees's own transform guidance: *"Remove (managed controllers) or update (operations center and client controllers) the license information."* A managed controller inherits its license from the operations center — its `jenkins.yaml` should have the `license:` block **deleted entirely**, not replaced with `${VAR}` placeholders. There's no per-controller license secret to wire up.

Also remove `unclassified.operationsCenterRootAction.connectionDetails` if present in the export — it's a live connection secret regenerated automatically at connect time, not durable config, and shouldn't be committed.

Bundle naming

Each controller's `bundle.yaml` `id` must be unique across **all** bundles in the repo (OC's and every controller's) — duplicate bundle names silently fail to sync.

Phase 3 — Commit the sanitized bundle(s) to git

Repo layout convention — one bundle per folder, **the folder's own name becomes the bundle ID** (this holds true regardless of how deeply nested the folder is, or how many other bundles share the repo):

```
None
cb-gitops/
└─ casc-bundles/
   └─ oc/                                # OC's own bundle, id: "oc"
      ├── bundle.yaml
      ├── jenkins.yaml
      └─ items.yaml                    # managed controller *definitions*
   └─ mc/
      ├── mc-ebs/                      # bundle id: "mc-ebs"
      │   ├── bundle.yaml
      │   ├── jenkins.yaml
      │   ├── plugins.yaml
      │   └─ plugin-catalog.yaml
      └─ items.yaml                    # this controller's jobs/folders
   └─ mc-<name>/                      # one folder per controller
```

```
Shell
git clone <your-repo-url>
mkdir -p casc-bundles/oc casc-bundles/mc/<controller-name>
# copy in the sanitized bundle.yaml / jenkins.yaml / plugins.yaml /
# plugin-catalog.yaml / items.yaml, per bundle, as applicable
git checkout -b feat/initial-casc-bundles
git add casc-bundles/
git commit -m "Add CasC bundles for OC and managed controllers"
git push -u origin feat/initial-casc-bundles
# open a PR at the URL git prints; merge via your git host's UI
# (direct pushes to main are commonly blocked by a repo policy hook)
```

Phase 4 — Create the Kubernetes Secret for externalized values (OC bundle only)

Shell

```
kubectl create secret generic cjoc-casc-secrets -n <namespace> \
  --from-literal=CJOC_LICENSE_CERT="$(cat /path/to/license-cert.pem)" \
  --from-literal=CJOC_LICENSE_KEY="$(cat /path/to/license-key.pem)" \
  --from-literal=SOME_CREDENTIAL_PASSWORD='<real value>'
```

Delete the local plaintext files immediately after. (Managed controller bundles don't need this for license, per Phase 2.5 — only if a controller's own `jenkins.yaml` has its own credential secrets to externalize.)

Phase 5 — Helm values: enable OC self-configuration

None

```
OperationsCenter:
  CasC:
    Enabled: true
    Retriever:
      Enabled: true
      scmRepo: https://github.com/<org>/<repo>.git
      scmBranch: main
      scmBundlePath: casc-bundles/oc
  ContainerEnvFrom:
    - secretRef:
        name: cjoc-casc-secrets
```

If the repo is **private**, add SCM auth via `OperationsCenter.CasC.Retriever.secrets` (`scmUsername/scmPassword` for HTTPS+PAT, or `sshConfig` referencing a secret with SSH key + `known_hosts`). Public repos need nothing here.

What this adds to the CJOC StatefulSet: a new init container (`casc-retriever-init-container`) that clones the bundle before `jenkins` starts (if this fails, CJOC does not boot at all); a new sidecar (`casc-retriever`) that polls the repo afterward; the `jenkins` container gains `-Dcore.casc.config.bundle=...` and `envFrom: secretRef: cjoc-casc-secrets`.

Phase 6 — Choose a mechanism to feed managed controllers their bundles

There are **two independent, non-interchangeable** mechanisms for this. Pick one per controller (or mix, but never conflate the two properties for the same controller — using the wrong one produces an identical crash signature either way, with no error pointing at the actual mistake).

Option A — Standalone CasC Controller Bundle Service

A separate `casc-bundle-service` microservice you stand up via Helm. Use this if you want bundle-serving decoupled from OC itself.

None

```
CascBundleService:
  enabled: true
  createConfig: true
```

This stands up a `casc-bundle-service` Deployment (2 replicas), Service, Ingress/HTTPRoute, RBAC, and CRD — but its config Secret is created **empty** by default. It does **not** wire up any controller automatically. After this Helm change:

1. Update the `casc-bundle-service-config` Secret with real connector info (pointing at the git repo/path for controller bundles).
2. Wire the controller to it — either in the CJOC UI (that controller's config → **Configuration as Code (CasC)** → **"CasC Bundles From CasC Bundle Service"** → pick the bundle by its `bundle.yaml description` → **Save**), or the CasC equivalent in the OC bundle's `items.yaml`:

None

```
- configurationAsCode:
  cascBundleServiceBundle: <base64("<connectorId>:<bundleId>")>
```

Compute the value with `printf '<connectorId>:<bundleId>' | base64 | tr -d '\n'`. Do **not** use `configurationAsCode.bundle: <bundle-id>` here — that property does nothing for this mechanism; `cascBundleServiceBundle` is what CJOC's object model actually reads (confirmed by exporting a controller already associated via the UI and reading back its live CasC).

Known gap — the Bundle Service never authorizes the controller's ServiceAccount. Steps 1–2 above are **not sufficient**. Symptom: the controller pod never becomes **Ready**, crash-loops with **Exit Code: 5** and a **503** readiness probe, retrying against the Bundle Service and getting HTTP 500s:

None

```
WARNING c.c.j.p.a.r.p.Downloader#registerError: Error response from server:

url=http://casc-bundle-service.<namespace>.svc.cluster.local/casc-bundle-service/api/v1/bundles/download/zip-bundle
status=500
```

Confirm the cause on the Bundle Service side:

Shell

```
kubectl -n <namespace> logs -l app=casc-bundle-service --tail=50 \
| grep "does not have access to any bundle"
```

WARNING K8sBundleReferenceManagement: Service account [system:serviceaccount:<namespace>:<controller>] does not have access to any bundle confirms it. **Root cause:** the Bundle Service authorizes each request by looking up the caller's Kubernetes ServiceAccount identity in a ConfigMap named **service-account-casc-bundle** in the OC's namespace. This ConfigMap ships **empty** on Helm install — no chart value populates it, and neither the UI association nor the CasC **cascBundleServiceBundle** property writes an entry into it, even though CJOC's own ServiceAccount has the RBAC to do so.

Workaround (undocumented internal object, not a supported long-term config path, but it persists across **helm upgrade** — repeat per controller):

Shell

```
# 1. ConfigMap key = base64 (no padding) of the controller's SA identity
KEY=$(printf 'system:serviceaccount:<namespace>:<controller>' | base64 | tr -d
'=\n')

# 2. bundleRef value = base64 (no padding) of "<connectorId>:<bundleId>"
# (same value as the cascBundleServiceBundle property above)
REF=$(printf '<connectorId>:<bundleId>' | base64 | tr -d '=\n')

# 3. Patch it in
```

```
kubectl -n <namespace> patch configmap service-account-casc-bundle --type merge -p \
{"data":{"$KEY":{"bundleRef":{"$REF":"","url":"","":""}}}}
```

Verify: the controller's next retry (or delete the pod to force one) should log **Performed the action download with accepted result** instead of **status=500**, and reach **1/1 Ready**. File a CloudBees support case citing this — it's a product gap, not a config mistake.

Option B — OC-hosted, git-backed bundle library (embedded, no separate service)

Use this if OC should serve controller bundles directly from the same git repo it already polls for its own config, with no extra Deployment.

OC's own `jenkins.yaml` (unclassified JCasc section) needs:

```
None
unclassified:
  bundleStorageService:
    activated: true
    checkOutTimeout: 600
    pollingPeriod: 120
    purgeOnDeactivation: false
    activeBundle:
      name: "<retriever-name>"
      polling: true
      retriever:
        SCM:
          defaultVersion: "main"
          scmSource:
            git:
              id: "<scm-source-id>"
              remote: "https://github.com/<org>/<repo>.git"
              traits:
                - "gitBranchDiscovery"
  cloudBeesCasCServer:
    disableRemoteValidation: false
    visibility: true
```

cloudBeesCasCServer.visibility is the load-bearing setting and it is easy to miss. If it's **false** (the value it may ship with), everything downstream *looks* fine and silently doesn't work:

- The retriever polls successfully, checks out the repo, and the bundle validates cleanly (all-INFO in `structural-validations.yaml`) — nothing signals a problem.
- The "CasC Bundles From Operations Center" dropdown on a controller's config page is simply **empty** — no error, no bundles listed.
- A controller wired to `configurationAsCode: {bundle: <id>}` (or the older `cascBundleSource: OC_CASC + bundle: <id>` form) crash-loops on boot:

None

```
SEVERE hudson.util.BootFailure#publish: Failed to initialize Jenkins
java.io.FileNotFoundException: Bundle file /var/casc-bundle/bundle-link.yaml not
found.
Caused: com.cloudbees.jenkins.cjp.installmanager.casc.InvalidBundleException:
Invalid configuration bundle: ...
```

There is no error anywhere pointing at `visibility` — confirm it directly on the OC pod:

Shell

```
kubectl -n <namespace> exec cjoc-0 -c jenkins -- \
  cat
/var/jenkins_home/com.cloudbees.opscenter.server.casc.BundleStorageConfiguration.x
ml
```

If `<visibility>false</visibility>`, set it to `true` via CasC (above) and restart OC (Phase 7) — don't flip it by hand in the UI, since that won't survive the next bundle reload from git.

Repo layout for this option: the retriever scans the **entire repo** (not one configured subdirectory) for any folder containing a `bundle.yaml` — see Phase 3's tree; nesting under `casc-bundles/mc/` works fine, the folder's own name is what matters.

Wire a controller to its bundle — in the OC bundle's `items.yaml`:

None

```
- kind: managedController
  name: mc-efs
  properties:
    - configurationAsCode:
      bundle: main/mc-efs
```

The value is `<branch>/<bundle-folder-name>`. OC resolves a bare `bundle: mc-efs` (no branch prefix) to this same qualified form internally using the retriever's `defaultVersion` as the implicit branch, but writing it out explicitly keeps git as the literal source of truth — confirm what's actually live by re-checking the item's config in the UI/export once it comes up.

Do **not** use `cascBundleSource: OC_CASC` — with `visibility: true` and this repo layout, the plain `bundle:` property is sufficient; `OC_CASC` implies a bundle uploaded directly to OC's own local store (a separate, unused path in this setup) and crash-loops identically to the `visibility: false` symptom above.

Phase 7 — Apply and watch it come up

Shell

```
helm get metadata <release> -n <namespace>    # confirm exact chart version first
helm upgrade <release> cloudbees/cloudbees-core \
  -n <namespace> \
  --version <current-chart-version> \
  --reuse-values \
  -f casc-values.yaml \
  --dry-run    # review the diff before applying for real
```

Drop `--dry-run` to apply. **This restarts CJOC**. Have a rollback plan ready:

Shell

```
helm rollback <release> <last-known-good-revision> -n <namespace>
```

For subsequent changes that are pure `items.yaml/jenkins.yaml` edits in git (no Helm value change) — e.g. adding a controller, changing its bundle association, scaling — merging to `main` alone is **not enough** to apply them. OC's own bundle only re-applies on an explicit reload, and `bundleUpdateTiming.automaticReload/automaticRestart` are typically `false` (check `/var/jenkins_home/bundle-update.properties` on the OC pod). The reliable way to force it:

Shell

```
kubectl -n <namespace> delete pod cjoc-0
kubectl -n <namespace> get pod cjoc-0 -w    # wait for Running, all containers
ready
```

This is disruptive (brief outage for the whole operations center — every managed controller loses contact with it, license renewal warnings appear in their logs) — expect to do it more than once per change:

- **A specific controller's item may not reprovision on the first restart.** Symptom: its logs show `Add CasC JAVA_OPTS for <name>` but never reach `PROVISIONING requested / PROVISIONED / STARTING` in OC's own logs (`kubectl logs cjoc-0 -c jenkins | grep <name>`), and no new Deployment/ReplicaSet appears. A second `kubectl delete pod cjoc-0` has reliably unstuck this.
- **A brand-new controller (never provisioned before)** may need an extra nudge beyond restart-triggered reconciliation — if still not progressing after a restart, try a manual "Provision" action on that controller's page in the OC UI.
- **Reprovisioning can leave two ReplicaSets for the same controller** — an old (stale, pre-fix) one still holding its pod/CPU allocation, and a new one that can't schedule because of it (`FailedScheduling: Insufficient cpu`). Confirm the old one predates the fix, then clear it:

Shell

```
kubectl -n <namespace> get replicaset | grep <controller-name>
kubectl -n <namespace> delete replicaset <old-stale-replicaset-name>
```

- If the CJOC pod itself crash-loops, check the init container first (runs before anything else), then the main container:

Shell

```
kubectl logs cjoc-0 -n <namespace> -c casc-retriever-init-container
kubectl logs cjoc-0 -n <namespace> -c jenkins --tail=1000 --previous >
/tmp/crash.log
grep -n "BootFailure\|Failed to initialize
Jenkins\|ConfiguratorException\|InvalidBundleException" /tmp/crash.log
```

The real root-cause line is `SEVERE hudson.util.BootFailure#publish: Failed to initialize Jenkins`, immediately followed by the actual exception — search for that exact phrase rather than trusting a small `--tail` window.

Verify a controller actually loaded its bundle — don't just trust `1/1 Running` (it can be Running on a stale/no bundle too, if you didn't clear an old ReplicaSet):

Shell

```
kubectl -n <namespace> logs <pod> --tail=300 | \
```

```
grep -n "BootFailure\\|FileNotFoundException\\|Creating items from CasC bundle
items section\\|jenkins is fully up"
```

Success: Creating items from CasC bundle items section + Jenkins is fully up and running, no BootFailure. Failure: the FileNotFoundException: Bundle file /var/casc-bundle/bundle-link.yaml not found signature from Phase 6, Option B.

Phase 8 — Verify

Shell

```
kubectl logs cjoc-0 -n <namespace> -c casc-retriever --tail=50 # confirm CJOC
picked up the bundle
```

Visit **Manage Jenkins** → **CloudBees Configuration as Code export and update** (or the CasC status page) on OC and on each managed controller to confirm the currently-applied bundle version/ID matches what you pushed, with no validation errors shown.

Phase 9 — Ongoing operations

9a. Bringing a brand-new controller under CasC (no prior config to sanitize)

If a controller has never been provisioned before, there's no existing config to export up front:

1. Scale it up via `items.yaml` (9b) with **no** `configurationAsCode` property yet — let it come up with whatever default/autogenerated config CloudBees CI gives it.
2. Once `1/1 Running`, export its live config (Phase 1, against its own URL).
3. Sanitize per Phase 2.5's rules (license, `operationsCenterRootAction`, `advisor`, drop `rbac.yaml`), fix `bundle.yaml`'s `id/description`.
4. Commit under `casc-bundles/mc/<id>/`, add the `configurationAsCode` property (Phase 6, whichever option you're using), push (Phase 3), apply (Phase 7).

9b. Scaling a managed controller via CasC

```
None
configuration:
  kubernetes:
    replication:
      config:
        maxReplicas: 0
        replicas: 0      # 0 = scaled down, 1 = up
        targetCPUUtilizationPercentage: 80
```

A controller with no `replication.config` block at all may simply never get provisioned — add the block explicitly to bring it up.

9c. Cluster capacity is zone-scoped, not just cluster-wide

A controller (or OC itself) whose PV is backed by a single-AZ storage class (e.g. plain EBS `gp3`) is **hard-pinned to that AZ** and can only ever schedule on nodes there:

```
Shell
kubectl get pv <pv-name> -o jsonpath='{.spec.nodeAffinity}'
kubectl get nodes -o
custom-columns=NAME:.metadata.name,ZONE:'.metadata.labels.topology\.kubernetes\.io
/zone'
```

FSx-OpenZFS- and EFS-backed PVs are typically **not** zone-pinned this way.

If cluster autoscaler can't add capacity (`NotTriggerScaleUp: ... max node group size reached`) and a zone-pinned pod is stuck `Pending` with `Insufficient cpu`, **freeing CPU cluster-wide is not sufficient** — it has to be freed in that specific AZ. Scaling down an unrelated controller in a *different* AZ won't help, even though the cluster-wide CPU math looks fine. Move a *non-zone-pinned* pod off the crowded AZ instead:

```
Shell
kubectl -n <namespace> delete pod <non-zone-pinned-pod>
# scheduler favors the less-loaded node; confirm it landed in the other zone:
kubectl -n <namespace> get pod <new-pod-name> -o wide
```

then retry/restart whatever was stuck.

Summary of gotchas (repeat offenders)

- Raw exports need sanitizing — license cert/key and credential passwords are the two hard blockers, always assume they're present for an OC/client controller bundle.
- `rbac.yaml`'s schema wants `groups/roles` even when unused — but adding them empty triggers a *different*, harder requirement (`RoleMatrixAuthorizationStrategy`). If the customer doesn't use matrix RBAC, just drop `rbac.yaml` from `bundle.yaml` entirely.
- `advisor: {}` empty blocks fail validation — decide explicitly whether to enable (`acceptToS + email`) or remove it, don't leave it as a stub.
- License/credentials get externalized via a Kubernetes Secret + `OperationsCenter.ContainerEnvFrom`, resolved through ordinary `${VAR}` CasC variable substitution — no separate Helm-native "upload your license" mechanism beyond the trial/evaluation flow.
- A managed controller's bundle isn't a smaller copy of the OC bundle — its `items.yaml` means jobs/folders (not controller definitions), it can have a `plugin-catalog.yaml` the OC bundle doesn't use, and its license block should be **removed**, not variable-substituted.
- The two controller-feeding mechanisms (Phase 6, Option A's `cascBundleServiceBundle` vs. Option B's `bundle: <branch>/<id>`) are **not interchangeable** — using the wrong property for whichever mechanism is actually configured produces the identical `bundle-link.yaml not found` crash either way, with nothing pointing at the real mistake.
- Option A's Bundle Service will 500 forever unless you manually patch the `service-account-casc-bundle` ConfigMap — CJOC never writes it itself despite having the RBAC to. This is a confirmed product gap, not a config mistake — file a support case.
- Option B's `cloudBeesCasCServer.visibility: false` silently breaks everything downstream — no error anywhere, just an empty bundle dropdown and a `bundle-link.yaml not found` crash loop, even though the bundle validates fine in the store.
- Applying `items.yaml` for the first time (or any subsequent change) can trigger unexpected reprovisioning of existing managed controllers, and that reprovisioning can stall or leave a stale ReplicaSet holding capacity — check cluster capacity before applying, and always verify the boot log (not just `Running` status) after.
- Zone-pinned storage (plain EBS) + a maxed-out node group in that specific AZ can starve OC itself, not just managed controllers — the fix is zone-aware capacity management, not just "free some CPU somewhere."